

מבוא למדעי הנתונים – עבודה 10

שאלה 1

```
# 1.a.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER
```

```
def load_dataset(file_name):  
    df_dataset = pd.read_csv(file_name)  
    return df_dataset
```

```
# 1.b.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER
```

```
def remove_missing_values(dataset):
    df_copy = dataset.copy()
    df_copy.dropna(inplace = True)
    return df_copy
```

```
# 1.c.  
# ----->>>>>>> RUN THIS CODE CELL <<<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER
```

```
def remove_duplicate_rows(dataset):
    dataset_cleaned = dataset.copy()
    dataset_cleaned.drop_duplicates(inplace = True, keep = 'first')
    return dataset_cleaned
```

```
# 1.d.
# ----->>>>>> RUN THIS CODE CELL <<<<<<<<-----
# ----- (run after implementation)
# == CODE TYPE: ASSISTANCE TO ANSWER
# --- Add assistance code here IF NEEDED:
```

```
### YOUR CODE HERE
###
cln_dataset['gender'].unique()
```

```
array(['female', 'male'], dtype=object)
```

```
# 1.d.  
# ----->>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# == CODE TYPE: ANSWER
```

```
def transfer_str_to_numeric_vals(dataset, str_col, id_col_to_remove):
    dataset_transferred = dataset.copy()
    replace_vals = {'male': 1, 'female': 0}
    dataset_transferred.replace(replace_vals, inplace = True)
    dataset_transferred.drop(labels = id_col_to_remove, axis = 1, inplace = True)
    return dataset_transferred
```

```
# 1.e.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER
```

```
def scale_dataset(dataset):
    dataset_scaled = dataset.copy()
    scaler = StandardScaler()
    return scaler.fit_transform(dataset_scaled)
```

שאלה 2 – אין שאלה, רק לנסות דברים

שאלה 3

```
# 3.b.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER  
  
def perform_hierarchical_clustering(dataset, num_clusters, linkage_val):  
    model = AgglomerativeClustering(n_clusters = num_clusters, linkage = linkage_val)  
    predicted_vals = model.fit_predict(dataset)  
    return model, predicted_vals
```

```
# 3.c.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER  
  
def perform_density_based_clustering(dataset, epsilon_val, minimum_samples_val):  
    model = DBSCAN(eps = epsilon_val, min_samples = minimum_samples_val)  
    model.fit(dataset)  
    predicted_vals = model.fit_predict(dataset)  
    return model, predicted_vals
```

```
## 4.a.  
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER  
  
import sys  
def get_best_init_params_for_k_means(dataset, num_clusters, init_options, n_init_options, rand_state):  
  
    INT_MAX = 2**31 - 1  
    best_score = INT_MAX  
    best_init_val = None  
    best_n_init_val = None  
  
    sum = 0  
    for init in init_options:  
        for n_init in n_init_options:  
            km, _ = perform_k_means(dataset, num_clusters, init, n_init, rand_state)  
            if km.inertia_ < best_score:  
                best_score = km.inertia_  
                best_init_val = init  
                best_n_init_val = n_init  
    return best_score, best_init_val, best_n_init_val
```

```
# 4.b.  
# ----->>>>> RUN THIS CODE CELL <<<<<<<-----  
# ----- (run after implementation)  
# === CODE TYPE: ANSWER  
  
def compare_number_of_clusters(dataset, num_cluster_options, init_val, n_init_val, rand_state):  
    lst_scores = []  
    for num_cluster in num_cluster_options:  
        km, _ = perform_k_means(dataset, num_cluster, init_val, n_init_val, rand_state)  
        lst_scores.append(km.inertia_)  
    return lst_scores
```

```

# 4.c.
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----
# ----- (run after implementation)
# === CODE TYPE: ANSWER

def get_best_num_of_clusters_for_k_means(dataset, num_cluster_options, init_val, n_init_val, rand_state):
    best_score = 0
    num_clusters = {}
    for i in num_cluster_options:
        _, y_pred = perform_k_means(dataset, i, init_val, n_init_val, rand_state)
        score = silhouette_score(dataset, y_pred)
        if score > best_score:
            best_score, num_clusters = score, i
    return best_score, num_clusters

```

שאלה 5

```

# 5.a.
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----
# ----- (run after implementation)
# === CODE TYPE: ANSWER

def get_best_linkage_method(dataset, num_clusters, linkage_options):
    best_linkage = ''
    best_score = 0
    for val in linkage_options:
        _, y_pred = perform_hierarchical_clustering(dataset, num_clusters, val)
        score = silhouette_score(dataset, y_pred)
        if score > best_score:
            best_score = score
            best_linkage = val
    return best_score, best_linkage

```

```

# 5.b.
# ----->>>>>> RUN THIS CODE CELL <<<<<<<-----
# ----- (run after implementation)
# === CODE TYPE: ANSWER

def get_best_params_for_dbscan(dataset, eps_options, min_samples_options):
    best_score, best_eps, best_min_samples = 0, 0, 0
    for i in eps_options:
        for j in min_samples_options:
            _, y_pred = perform_density_based_clustering(dataset, i, j)
            score = silhouette_score(dataset, y_pred)
            if score > best_score:
                best_score, best_eps, best_min_samples = score, i, j
    return best_score, best_eps, best_min_samples

```

